THOMAS MORE · ELECTRONICS-ICT

SMART HOME HUB

Firmware Code Guide

Function-by-function explanation of the STM32F746 smart-home firmware

Student	Mahmoud Mostafa
Target	STM32F746BGT6 · STM32CubeIDE / HAL
Source baseline	SmartHome_Weather_GUI_CubeIDE
Functions documented	141
Build state	124 active · 17 conditional/diagnostic
Revision	1.0 · 15 August 2026

SCOPE

This guide documents every function defined in main.c and qspi_assets.c. Generated bitmap/font arrays and STM32 HAL library internals contain no project functions and are described only at module level.

1 How the firmware is organized

A map from startup to sensing, safety, presentation and cloud services

The firmware is a cooperative, super-loop application. It has no RTOS: main() owns the schedule and invokes short local tasks at fixed HAL tick intervals. The only intentionally blocking work is ESP-01 AT/HTTP communication; flame safety is sampled inside those waits and the buzzer is restored immediately afterward.

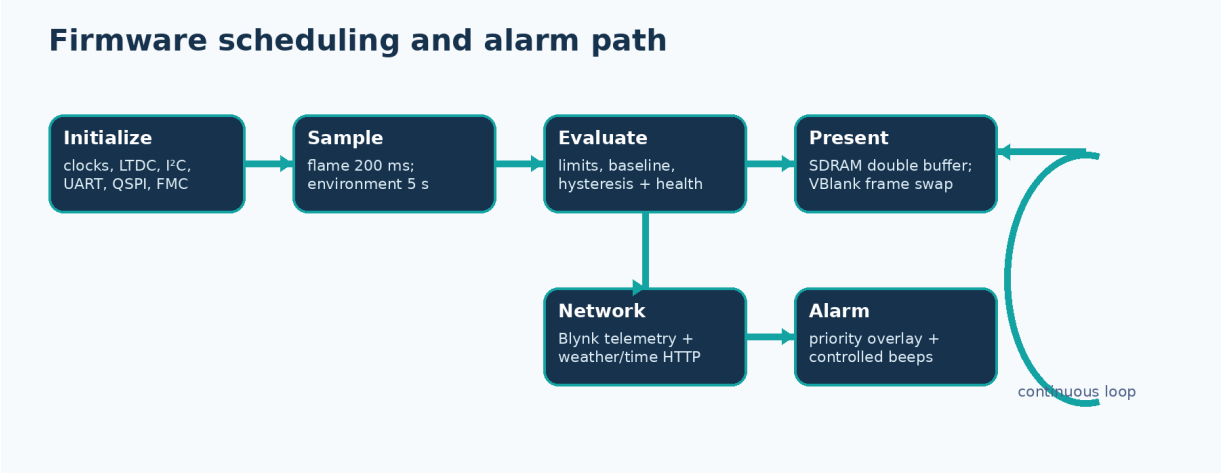

Figure 1. Main-loop scheduling and the local-safety priority path.

1.1 Source modules

Module	Role	Function definitions
Core/Src/main.c	Application, drivers, GUI, scheduling and CubeMX initializers	109
Core/Src/qspi_assets.c	W25Q128 asset installer, CRC validation and memory-mapped accessors	32
gui_modern_assets.c	Generated font and icon byte arrays; no functions	0
weather_backgrounds.c	Generated weather background byte arrays; no functions	0

1.2 Important data ownership

Data	Owner / address	Why it matters
GUI frame 0	SDRAM 0xC0000000	Visible or drawing L8 frame, depending on current swap state
GUI frame 1	SDRAM 0xC0040000	Second full-screen L8 frame for tear-free updates
Weather/fonts/icons	QSPI 0x90000000 window	Persistent, CRC-checked read-only assets after startup
SensorValues	Internal SRAM	Latest validated local readings and alarm state
InternetWeather	Internal SRAM	Detected location, time sync, weather cache and selected theme
UNITS		

Data	Owner / address	Why it matters
Temperatures and relative humidity use integer tenths (for example 302 means 30.2 °C). Pressure uses hPa×10. This avoids float-dependent logic in the safety and formatting paths.		

2 Complete function reference

Every function defined by the project, including conditional diagnostics and installer-only helpers

Entries are grouped by subsystem rather than source order. “Active” means compiled in the normal application. Conditional entries remain useful documentation but are excluded by #if 0, QSPI_ASSET_INSTALLER or USE_FULL_ASSERT in a normal release build.

2.1 Startup, scheduler and fault handling

6 functions in this subsystem.

Function	What it does / how it works	Interface	Dependencies
MPU_FixExternalMemory() main.c:2394–2446 Active	Re-applies safe MPU regions for FMC/QSPI registers, 8 MiB SDRAM and the 16 MiB memory-mapped QSPI window. It coordinates HAL_MPU_Disable(), HAL_MPU_ConfigRegion(), HAL_MPU_Enable().	**In:** None. **Out:** No return value; the function acts through hardware, buffers or state.	Calls: HAL_MPU_Disable(), HAL_MPU_ConfigRegion(), HAL_MPU_Enable(), __DSB(), __ISB(); Uses: LTDC.
main() main.c:5036–5467 Active	Initializes memory, peripherals, sensors and Wi-Fi, then runs the cooperative scheduler for sampling, alarms, GUI, weather and Blynk uploads. It coordinates MPU_Config(), HAL_Init(), MPU_FixExternalMemory(), SystemClock_Config(), MX_GPIO_Init(), MX_LTDC_Init(). The implementation contains 31 explicit decision/error checks and 1 loop.	**In:** None. **Out:** Returns a numeric result of type int.	Calls: MPU_Config(), HAL_Init(), MPU_FixExternalMemory(), SystemClock_Config(), MX_GPIO_Init(), MX_LTDC_Init(), MX_USART1_UART_Init(), MX_ADC1_Init(); Uses: LTDC, QUADSPI/W25Q128, USART3 debug, buzzer GPIO.
SystemClock_Config() main.c:5474–5524 Active	Configures HSE/PLL clocks for the 200 MHz Cortex-M7 system and peripheral clock domains. It coordinates HAL_PWR_EnableBkUpAccess(), HAL_RCC_OscConfig(), Error_Handler(), HAL_PWREx_EnableOverDrive(), HAL_RCC_ClockConfig(). The implementation contains 3 explicit decision/error checks.	**In:** None. **Out:** No return value; the function acts through hardware, buffers or state.	Calls: HAL_PWR_EnableBkUpAccess(), __HAL_RCC_PWR_CLK_ENABLE(), __HAL_PWR_VOLTAGESCALING_CONFIG(), HAL_RCC_OscConfig(), Error_Handler(), HAL_PWREx_EnableOverDrive(), HAL_RCC_ClockConfig().
MPU_Config() main.c:6061–6111 Active	Installs the base MPU policy before HAL startup, including protection required by the Cortex-M7 memory system. It coordinates HAL_MPU_Disable(), HAL_MPU_ConfigRegion(), HAL_MPU_Enable().	**In:** None. **Out:** No return value; the function acts through hardware, buffers or state.	Calls: HAL_MPU_Disable(), HAL_MPU_ConfigRegion(), HAL_MPU_Enable().

Function	What it does / how it works	Interface	Dependencies
Error_Handler() main.c:6118–6126 Active	Disables interrupts and traps execution after an unrecoverable initialization or HAL error. The implementation contains 1 loop.	In: None. Out: No return value; the function acts through hardware, buffers or state.	Calls: __disable_irq().
assert_failed() main.c:6136–6141 USE_FULL_ASSERT only	Optional full-assert callback that exposes the source file and line of a failed HAL assertion to the debugger.	In: uint8_t *file, uint32_t line Out: No return value; the function acts through hardware, buffers or state.	Pure/local helper; no direct peripheral dependency detected..

2.2 CubeMX peripheral initialization

10 functions in this subsystem.

Function	What it does / how it works	Interface	Dependencies
MX_ADC1_Init() main.c:5532–5577 Active	Configures ADC1 for the PA1 flame analog channel. It coordinates HAL_ADC_Init(), Error_Handler(), HAL_ADC_ConfigChannel(). The implementation contains 2 explicit decision/error checks.	In: None. Out: No return value; the function acts through hardware, buffers or state.	Calls: HAL_ADC_Init(), Error_Handler(), HAL_ADC_ConfigChannel(); Uses: ADC1/PA1.
MX_I2C1_Init() main.c:5585–5625 Active	Configures I2C1 retained for the display/touch hardware path. It coordinates HAL_I2C_Init(), Error_Handler(), HAL_I2CEx_ConfigAnalogFilter(), HAL_I2CEx_ConfigDigitalFilter(). The implementation contains 3 explicit decision/error checks.	In: None. Out: No return value; the function acts through hardware, buffers or state.	Calls: HAL_I2C_Init(), Error_Handler(), HAL_I2CEx_ConfigAnalogFilter(), HAL_I2CEx_ConfigDigitalFilter().
MX_I2C3_Init() main.c:5633–5673 Active	Configures I2C3 for the BME688 bus. It coordinates HAL_I2C_Init(), Error_Handler(), HAL_I2CEx_ConfigAnalogFilter(), HAL_I2CEx_ConfigDigitalFilter(). The implementation contains 3 explicit decision/error checks.	In: None. Out: No return value; the function acts through hardware, buffers or state.	Calls: HAL_I2C_Init(), Error_Handler(), HAL_I2CEx_ConfigAnalogFilter(), HAL_I2CEx_ConfigDigitalFilter(); Uses: I2C3/BME688.
MX_I2C4_Init() main.c:5681–5721 Active	Configures I2C4 for the SCD41 bus. It coordinates HAL_I2C_Init(), Error_Handler(), HAL_I2CEx_ConfigAnalogFilter(), HAL_I2CEx_ConfigDigitalFilter(). The implementation contains 3 explicit decision/error checks.	In: None. Out: No return value; the function acts through hardware, buffers or state.	Calls: HAL_I2C_Init(), Error_Handler(), HAL_I2CEx_ConfigAnalogFilter(), HAL_I2CEx_ConfigDigitalFilter(); Uses: I2C4/SCD41.
MX_LTDC_Init() main.c:5729–5817 Active	Programs LTDC timing, polarity, pixel clock interface and initial layers for the 480×272 RGB panel. It coordinates HAL_LTDC_Init(), Error_Handler(), HAL_LTDC_ConfigLayer(). The implementation contains 4 explicit decision/error checks.	In: None. Out: No return value; the function acts through hardware, buffers or state.	Calls: HAL_LTDC_Init(), Error_Handler(), HAL_LTDC_ConfigLayer(); Uses: LTDC.

Function	What it does / how it works	Interface	Dependencies
MX_QUADSPI_Init() main.c:5825–5854 Active	Configures the STM32 QUADSPI peripheral for the W25Q128 flash geometry and clock. It coordinates HAL_QSPI_Init(), Error_Handler(). The implementation contains 1 explicit decision/error check.	In: None. Out: No return value; the function acts through hardware, buffers or state.	Calls: HAL_QSPI_Init(), Error_Handler(); Uses: LTDC, QUADSPI/W25Q128.
MX_USART1_UART_Init() main.c:5862–5889 Active	Configures USART1 for ESP-01 AT communication. It coordinates HAL_UART_Init(), Error_Handler(). The implementation contains 1 explicit decision/error check.	In: None. Out: No return value; the function acts through hardware, buffers or state.	Calls: HAL_UART_Init(), Error_Handler(); Uses: USART1/ESP-01.
MX_USART3_UART_Init() main.c:5897–5924 Active	Configures USART3 as the firmware diagnostic/debug UART. It coordinates HAL_UART_Init(), Error_Handler(). The implementation contains 1 explicit decision/error check.	In: None. Out: No return value; the function acts through hardware, buffers or state.	Calls: HAL_UART_Init(), Error_Handler(); Uses: USART3 debug.
MX_FMC_Init() main.c:5928–5972 Active	Configures FMC SDRAM Bank 1 bus width, row/column geometry and timing for the IS42S16400J. It coordinates HAL_SDRAM_Init(), Error_Handler(). The implementation contains 1 explicit decision/error check.	In: None. Out: No return value; the function acts through hardware, buffers or state.	Calls: HAL_SDRAM_Init(), Error_Handler(); Uses: FMC/SDRAM.
MX_GPIO_Init() main.c:5980–6052 Active	Enables GPIO clocks, establishes safe output levels and configures buzzer, flame, ESP and control pins. It coordinates HAL_GPIO_WritePin(), HAL_GPIO_Init(), HAL_Delay().	In: None. Out: No return value; the function acts through hardware, buffers or state.	Calls: __HAL_RCC_GPIOE_CLK_ENABLE(), __HAL_RCC_GPIOC_CLK_ENABLE(), __HAL_RCC_GPIOI_CLK_ENABLE(), __HAL_RCC_GPIOF_CLK_ENABLE(), __HAL_RCC_GPIOH_CLK_ENABLE(), __HAL_RCC_GPIOA_CLK_ENABLE(), __HAL_RCC_GPIOB_CLK_ENABLE(), __HAL_RCC_GPIOJ_CLK_ENABLE(); Uses: buzzer GPIO.

2.3 External memory and QSPI assets

43 functions in this subsystem.

Function	What it does / how it works	Interface	Dependencies
SDRAM_Startup() main.c:1920–1971 Active	Sends the JEDEC SDRAM power-up command sequence and programs the proven 66.7 MHz refresh count. It coordinates HAL_SDRAM_SendCommand(), HAL_Delay(), HAL_SDRAM_ProgramRefreshRate(). The implementation contains 4 explicit decision/error checks.	In: None. Out: HAL_OK on success; HAL_ERROR (or another HAL status) is propagated on failure.	Calls: HAL_SDRAM_SendCommand(), HAL_Delay(), HAL_SDRAM_ProgramRefreshRate(); Uses: FMC/SDRAM.

Function	What it does / how it works	Interface	Dependencies
SDRAM_Test() main.c:1982–2051 Disabled diagnostic (#if 0)	Destructively writes and verifies two complementary pseudo-random patterns across all 8 MiB while caches are disabled. The implementation contains 6 explicit decision/error checks and 3 loops.	**In:** None. **Out:** true when the operation/result is valid; false on unavailable data, validation failure or I/O error.	Calls: SCB_CleanDCache(), SCB_DisableDCache(), __DSB(), SCB_InvalidateDCache(), SCB_EnableDCache().
QSPI_CommandDefaults() main.c:2059–2072 Disabled diagnostic (#if 0)	Legacy diagnostic copy: initializes a QSPI command descriptor with safe single-line defaults before a helper specializes it.	**In:** QSPI_CommandTypeDef *cmd, uint8_t instruction **Out:** No return value; the function acts through hardware, buffers or state.	Pure/local helper; no direct peripheral dependency detected..
QSPI_CommandOnly() main.c:2075–2081 Disabled diagnostic (#if 0)	Legacy diagnostic copy: sends a one-byte flash instruction that has no address or data phase. It coordinates QSPI_CommandDefaults(), HAL_QSPI_Command().	**In:** uint8_t instruction **Out:** HAL_OK on success; HAL_ERROR (or another HAL status) is propagated on failure.	Calls: QSPI_CommandDefaults(), HAL_QSPI_Command(); Uses: QUADSPI/W25Q128.
QSPI_ReadRegister() main.c:2085–2096 Disabled diagnostic (#if 0)	Legacy diagnostic copy: reads one W25Q128 status/register byte through a single-line data phase. It coordinates QSPI_CommandDefaults(), HAL_QSPI_Command(), HAL_QSPI_Receive(). The implementation contains 1 explicit decision/error check.	**In:** uint8_t instruction, uint8_t *value **Out:** HAL_OK on success; HAL_ERROR (or another HAL status) is propagated on failure.	Calls: QSPI_CommandDefaults(), HAL_QSPI_Command(), HAL_QSPI_Receive(); Uses: QUADSPI/W25Q128.
QSPI_PollStatus() main.c:2101–2123 Disabled diagnostic (#if 0)	Legacy diagnostic copy: uses HAL automatic polling to wait until masked W25Q128 status bits match an expected value. It coordinates QSPI_CommandDefaults(), HAL_QSPI_AutoPolling().	**In:** uint8_t match, uint8_t mask, uint32_t timeout **Out:** HAL_OK on success; HAL_ERROR (or another HAL status) is propagated on failure.	Calls: QSPI_CommandDefaults(), HAL_QSPI_AutoPolling(); Uses: QUADSPI/W25Q128.
QSPI_WaitReady() main.c:2126–2129 Disabled diagnostic (#if 0)	Legacy diagnostic copy: waits for the W25Q128 BUSY bit to clear. It coordinates QSPI_PollStatus().	**In:** uint32_t timeout **Out:** HAL_OK on success; HAL_ERROR (or another HAL status) is propagated on failure.	Calls: QSPI_PollStatus().
QSPI_WriteEnable() main.c:2132–2138 Disabled diagnostic (#if 0)	Legacy diagnostic copy: issues Write Enable and confirms the write-enable latch before erase/program operations. It coordinates QSPI_CommandOnly(), QSPI_PollStatus(). The implementation contains 1 explicit decision/error check.	**In:** None. **Out:** HAL_OK on success; HAL_ERROR (or another HAL status) is propagated on failure.	Calls: QSPI_CommandOnly(), QSPI_PollStatus().

Function	What it does / how it works	Interface	Dependencies
QSPI_EnableQuadMode() main.c:2141–2175 Disabled diagnostic (#if 0)	Legacy diagnostic copy: reads status register 2, sets the QE bit when needed and verifies that four-line transfers are enabled. It coordinates QSPI_ReadRegister(), QSPI_WriteEnable(), QSPI_CommandDefaults(), HAL_QSPI_Command(), HAL_QSPI_Transmit(), QSPI_WaitReady(). The implementation contains 7 explicit decision/error checks.	**In:** None. **Out:** HAL_OK on success; HAL_ERROR (or another HAL status) is propagated on failure.	Calls: QSPI_ReadRegister(), QSPI_WriteEnable(), QSPI_CommandDefaults(), HAL_QSPI_Command(), HAL_QSPI_Transmit(), QSPI_WaitReady(); Uses: QUADSPI/W25Q128.
QSPI_Test() main.c:2179–2307 Disabled diagnostic (#if 0)	Legacy diagnostic copy: destructive legacy flash diagnostic that checks JEDEC ID, erases one sector, programs a pattern and verifies a quad readback. It coordinates QSPI_CommandOnly(), HAL_Delay(), QSPI_CommandDefaults(), HAL_QSPI_Command(), HAL_QSPI_Receive(), QSPI_EnableQuadMode(). The implementation contains 13 explicit decision/error checks and 1 loop.	**In:** None. **Out:** true when the operation/result is valid; false on unavailable data, validation failure or I/O error.	Calls: QSPI_CommandOnly(), HAL_Delay(), QSPI_CommandDefaults(), HAL_QSPI_Command(), HAL_QSPI_Receive(), QSPI_EnableQuadMode(), QSPI_WriteEnable(), QSPI_WaitReady(); Uses: QUADSPI/W25Q128.
MemoryTest_Show() main.c:2312–2369 Disabled diagnostic (#if 0)	Displays SDRAM/QSPI diagnostic results, including the first failing SDRAM address and expected/actual words. It coordinates LCD_ClearTextLayer(), LCD_DrawString(), LCD_PresentTextLayer(). The implementation contains 1 explicit decision/error check.	**In:** bool sdram_ok, bool qspi_ok **Out:** No return value; the function acts through hardware, buffers or state.	Calls: LCD_ClearTextLayer(), LCD_DrawString(), LCD_PresentTextLayer().
QSPI_CommandDefaults() qspi_assets.c:155–167 Active	Initializes a QSPI command descriptor with safe single-line defaults before a helper specializes it.	**In:** QSPI_CommandTypeDef *command, uint8_t instruction **Out:** No return value; the function acts through hardware, buffers or state.	Pure/local helper; no direct peripheral dependency detected..
QSPI_CommandOnly() qspi_assets.c:170–175 Active	Sends a one-byte flash instruction that has no address or data phase. It coordinates QSPI_CommandDefaults(), HAL_QSPI_Command().	**In:** uint8_t instruction **Out:** HAL_OK on success; HAL_ERROR (or another HAL status) is propagated on failure.	Calls: QSPI_CommandDefaults(), HAL_QSPI_Command().
QSPI_ReadRegister() qspi_assets.c:179–190 Active	Reads one W25Q128 status/register byte through a single-line data phase. It coordinates QSPI_CommandDefaults(), HAL_QSPI_Command(), HAL_QSPI_Receive(). The implementation contains 1 explicit decision/error check.	**In:** uint8_t instruction, uint8_t *value **Out:** HAL_OK on success; HAL_ERROR (or another HAL status) is propagated on failure.	Calls: QSPI_CommandDefaults(), HAL_QSPI_Command(), HAL_QSPI_Receive().

Function	What it does / how it works	Interface	Dependencies
QSPI_PollStatus() qspi_assets.c:195–215 Active	Uses HAL automatic polling to wait until masked W25Q128 status bits match an expected value. It coordinates QSPI_CommandDefaults(), HAL_QSPI_AutoPolling().	In: uint8_t match, uint8_t mask, uint32_t timeout Out: HAL_OK on success; HAL_ERROR (or another HAL status) is propagated on failure.	Calls: QSPI_CommandDefaults(), HAL_QSPI_AutoPolling().
QSPI_WaitReady() qspi_assets.c:218–220 Active	Waits for the W25Q128 BUSY bit to clear. It coordinates QSPI_PollStatus().	In: uint32_t timeout Out: HAL_OK on success; HAL_ERROR (or another HAL status) is propagated on failure.	Calls: QSPI_PollStatus().
QSPI_WriteEnable() qspi_assets.c:223–228 Active	Issues Write Enable and confirms the write-enable latch before erase/program operations. It coordinates QSPI_CommandOnly(), QSPI_PollStatus(). The implementation contains 1 explicit decision/error check.	In: None. Out: HAL_OK on success; HAL_ERROR (or another HAL status) is propagated on failure.	Calls: QSPI_CommandOnly(), QSPI_PollStatus().
QSPI_EnableQuadMode() qspi_assets.c:231–263 Active	Reads status register 2, sets the QE bit when needed and verifies that four-line transfers are enabled. It coordinates QSPI_ReadRegister(), QSPI_WriteEnable(), QSPI_CommandDefaults(), HAL_QSPI_Command(), HAL_QSPI_Transmit(), QSPI_WaitReady(). The implementation contains 7 explicit decision/error checks.	In: None. Out: HAL_OK on success; HAL_ERROR (or another HAL status) is propagated on failure.	Calls: QSPI_ReadRegister(), QSPI_WriteEnable(), QSPI_CommandDefaults(), HAL_QSPI_Command(), HAL_QSPI_Transmit(), QSPI_WaitReady().
QSPI_Read() qspi_assets.c:268–285 Active	Reads an arbitrary asset range with quad-output fast read into an STM32 buffer. It coordinates QSPI_CommandDefaults(), HAL_QSPI_Command(), HAL_QSPI_Receive(). The implementation contains 2 explicit decision/error checks.	In: uint32_t address, uint8_t *destination, uint32_t length Out: HAL_OK on success; HAL_ERROR (or another HAL status) is propagated on failure.	Calls: QSPI_CommandDefaults(), HAL_QSPI_Command(), HAL_QSPI_Receive().
QSPI_EraseAssetRegion() qspi_assets.c:290–312 Installer build only	Installer-only routine that erases every sector occupied by the asset store. It coordinates QSPI_WriteEnable(), QSPI_CommandDefaults(), HAL_QSPI_Command(), QSPI_WaitReady(). The implementation contains 3 explicit decision/error checks and 1 loop.	In: None. Out: HAL_OK on success; HAL_ERROR (or another HAL status) is propagated on failure.	Calls: QSPI_WriteEnable(), QSPI_CommandDefaults(), HAL_QSPI_Command(), QSPI_WaitReady().
QSPI_Program() qspi_assets.c:317–358 Installer build only	Installer-only paged programmer that respects 256-byte W25Q128 page boundaries and waits after every page. It coordinates QSPI_WriteEnable(), QSPI_CommandDefaults(), HAL_QSPI_Command(), HAL_QSPI_Transmit(), QSPI_WaitReady(). The implementation contains 5 explicit decision/error checks and 1 loop.	In: uint32_t address, const uint8_t *source, uint32_t length Out: HAL_OK on success; HAL_ERROR (or another HAL status) is propagated on failure.	Calls: QSPI_WriteEnable(), QSPI_CommandDefaults(), HAL_QSPI_Command(), HAL_QSPI_Transmit(), QSPI_WaitReady().

Function	What it does / how it works	Interface	Dependencies
CRC32_Update() qspi_assets.c:365–378 Active	Updates the asset CRC-32 accumulator over a byte sequence. The implementation contains 2 loops.	**In:** uint32_t crc, const uint8_t *data, uint32_t length **Out:** Returns a numeric result of type uint32_t.	Pure/local helper; no direct peripheral dependency detected..
CRC32_Buffer() qspi_assets.c:381–383 Active	Calculates a complete CRC-32 for one in-memory buffer. It coordinates CRC32_Update().	**In:** const uint8_t *data, uint32_t length **Out:** Returns a numeric result of type uint32_t.	Calls: CRC32_Update().
QSPI_SourceFontCRC() qspi_assets.c:388–399 Installer build only	Installer-only CRC calculation over the built-in legacy font source arrays. It coordinates CRC32_Update().	**In:** None. **Out:** Returns a numeric result of type uint32_t.	Calls: CRC32_Update().
QSPI_SourceWeatherCRC() qspi_assets.c:405–435 Installer build only	Installer-only CRC calculation over all compiled weather background assets. It coordinates CRC32_Update(). The implementation contains 3 loops.	**In:** None. **Out:** Returns a numeric result of type uint32_t.	Calls: CRC32_Update().
QSPI_HeaderMatchesInstallerSource() qspi_assets.c:439–448 Installer build only	Checks whether the installed header version, sizes and CRCs already match the compiled installer sources. It coordinates QSPI_SourceWeatherCRC(), CRC32_Buffer(), QSPI_SourceFontCRC().	**In:** const QSPI_AssetHeader *header **Out:** true when the operation/result is valid; false on unavailable data, validation failure or I/O error.	Calls: QSPI_SourceWeatherCRC(), CRC32_Buffer(), QSPI_SourceFontCRC().
QSPI_CRCRegion() qspi_assets.c:455–477 Active	Streams a QSPI region in chunks and computes its CRC without requiring a region-sized RAM buffer. It coordinates QSPI_Read(), CRC32_Update(). The implementation contains 2 explicit decision/error checks and 1 loop.	**In:** uint32_t address, uint32_t length, uint32_t *result **Out:** HAL_OK on success; HAL_ERROR (or another HAL status) is propagated on failure.	Calls: QSPI_Read(), CRC32_Update().
QSPI_HeaderIsValid() qspi_assets.c:480–509 Active	Validates asset magic, version, offsets, lengths and the header's own CRC before any pointer is exposed. It coordinates CRC32_Buffer(). The implementation contains 2 explicit decision/error checks.	**In:** const QSPI_AssetHeader *header **Out:** true when the operation/result is valid; false on unavailable data, validation failure or I/O error.	Calls: CRC32_Buffer().

Function	What it does / how it works	Interface	Dependencies
QSPI_VerifyAssets() qspi_assets.c:512–551 Active	Recomputes CRCs for background, palette, legacy font and modern asset regions and compares them with the header. It coordinates QSPI_HeaderIsValid(), QSPI_CRCRegion(). The implementation contains 5 explicit decision/error checks.	In: const QSPI_AssetHeader *header Out: HAL_OK on success; HAL_ERROR (or another HAL status) is propagated on failure.	Calls: QSPI_HeaderIsValid(), QSPI_CRCRegion().
QSPI_InstallAssets() qspi_assets.c:556–678 Installer build only	Erases and programs the header, weather images, palette, fonts and icons, then verifies the completed flash image. It coordinates QSPI_EraseAssetRegion(), QSPI_Program(), QSPI_SourceWeatherCRC(), CRC32_Buffer(), QSPI_SourceFontCRC(), QSPI_Read(). The implementation contains 7 explicit decision/error checks and 3 loops.	In: QSPI_AssetHeader *header Out: HAL_OK on success; HAL_ERROR (or another HAL status) is propagated on failure.	Calls: QSPI_EraseAssetRegion(), QSPI_Program(), QSPI_SourceWeatherCRC(), CRC32_Buffer(), QSPI_SourceFontCRC(), QSPI_Read(), QSPI_VerifyAssets().
QSPI_EnterMemoryMappedMode() qspi_assets.c:683–696 Active	Places the W25Q128 in continuous memory-mapped quad-read mode at 0x90000000. It coordinates QSPI_CommandDefaults(), HAL_QSPI_MemoryMapped().	In: None. Out: HAL_OK on success; HAL_ERROR (or another HAL status) is propagated on failure.	Calls: QSPI_CommandDefaults(), HAL_QSPI_MemoryMapped().
QSPI_Assets_Init() qspi_assets.c:700–787 Active	Resets and identifies the flash, enables quad mode, installs assets when requested, validates CRCs and enters memory-mapped mode. It coordinates QSPI_CommandOnly(), HAL_Delay(), QSPI_CommandDefaults(), HAL_QSPI_Command(), HAL_QSPI_Receive(), QSPI_EnableQuadMode(). The implementation contains 11 explicit decision/error checks.	In: QSPI_HandleTypeDef *hqspi, bool *installed_now Out: HAL_OK on success; HAL_ERROR (or another HAL status) is propagated on failure.	Calls: QSPI_CommandOnly(), HAL_Delay(), QSPI_CommandDefaults(), HAL_QSPI_Command(), HAL_QSPI_Receive(), QSPI_EnableQuadMode(), QSPI_Read(), QSPI_HeaderIsValid(); Uses: LTDC, QUADSPI/W25Q128.
QSPI_Assets_BackgroundL8() qspi_assets.c:790–792 Active	Compatibility accessor returning the clear-day weather background pointer. It coordinates QSPI_Assets_WeatherBackgroundL8().	In: None. Out: Pointer into the validated memory-mapped asset store, or NULL when invalid.	Calls: QSPI_Assets_WeatherBackgroundL8().
QSPI_Assets_WeatherBackgroundL8() qspi_assets.c:795–807 Active	Returns the memory-mapped pointer to one selected 480×272 L8 weather background. The implementation contains 2 explicit decision/error checks.	In: QSPI_WeatherTheme theme Out: Pointer into the validated memory-mapped asset store, or NULL when invalid.	Pure/local helper; no direct peripheral dependency detected..
QSPI_Assets_CLUT() qspi_assets.c:810–817 Active	Returns the memory-mapped 256-entry LTDC colour lookup table. The implementation contains 1 explicit decision/error check.	In: None. Out: Pointer into the validated memory-mapped asset store, or NULL when invalid.	Pure/local helper; no direct peripheral dependency detected..

Function	What it does / how it works	Interface	Dependencies
QSPI_Assets_GetGlyph5x7() qspi_assets.c:820–846 Active	Looks up a supported legacy character and returns its seven-row bitmap. The implementation contains 3 explicit decision/error checks and 1 loop.	**In:** char character **Out:** Pointer into the validated memory-mapped asset store, or NULL when invalid.	Pure/local helper; no direct peripheral dependency detected..
QSPI_ReadLE16() qspi_assets.c:849–852 Active	Decodes a little-endian 16-bit value from a packed modern-asset byte stream.	**In:** const uint8_t *data **Out:** Returns a numeric result of type uint16_t.	Pure/local helper; no direct peripheral dependency detected..
QSPI_ReadLE32() qspi_assets.c:855–860 Active	Decodes a little-endian 32-bit value from a packed modern-asset byte stream.	**In:** const uint8_t *data **Out:** Returns a numeric result of type uint32_t.	Pure/local helper; no direct peripheral dependency detected..
QSPI_ModernAssetBase() qspi_assets.c:863–888 Active	Validates the modern-font sub-header and returns its memory-mapped base pointer. It coordinates QSPI_ReadLE32(), QSPI_ReadLE16(). The implementation contains 2 explicit decision/error checks.	**In:** None. **Out:** Pointer into the validated memory-mapped asset store, or NULL when invalid.	Calls: QSPI_ReadLE32(), QSPI_ReadLE16().
QSPI_Assets_GetModernGlyph() qspi_assets.c:893–941 Active	Locates a character descriptor for a selected font size and fills a QSPI_FontGlyph view of its bitmap. It coordinates QSPI_ModernAssetBase(), QSPI_ReadLE32(). The implementation contains 5 explicit decision/error checks.	**In:** QSPI_FontSize size, char character, QSPI_FontGlyph *glyph **Out:** true when the operation/result is valid; false on unavailable data, validation failure or I/O error.	Calls: QSPI_ModernAssetBase(), QSPI_ReadLE32().
QSPI_Assets_ModernLineHeight() qspi_assets.c:944–951 Active	Returns the recommended line height for small, medium or large modern font. It coordinates QSPI_ModernAssetBase(). The implementation contains 1 explicit decision/error check.	**In:** QSPI_FontSize size **Out:** Returns a numeric result of type uint8_t.	Calls: QSPI_ModernAssetBase().
QSPI_Assets_GetIcon() qspi_assets.c:954–971 Active	Returns the bitmap address of a selected 20×20 icon after validating the requested index and asset bounds. It coordinates QSPI_ModernAssetBase(), QSPI_ReadLE32(). The implementation contains 2 explicit decision/error checks.	**In:** QSPI_Icon icon **Out:** Pointer into the validated memory-mapped asset store, or NULL when invalid.	Calls: QSPI_ModernAssetBase(), QSPI_ReadLE32().
QSPI_Assets_LastError() qspi_assets.c:974–976 Active	Returns the last detailed asset initialization/validation error code.	**In:** None. **Out:** Returns QSPI_AssetError.	Pure/local helper; no direct peripheral dependency detected..

2.4 Display, drawing and GUI

30 functions in this subsystem.

Function	What it does / how it works	Interface	Dependencies
LCD_SetBackground() main.c:200–211 Active	Disables the framebuffer layers and sets the LTDC controller's solid RGB background colour.	**In:** uint8_t red, uint8_t green, uint8_t blue **Out:** No return value; the function acts through hardware, buffers or state.	Uses: LTDC.
LCD_DrawPixel() main.c:256–261 Active	Writes one palette index to the current SDRAM back buffer after checking the display bounds. The implementation contains 1 explicit decision/error check.	**In:** uint16_t x, uint16_t y, uint16_t colour **Out:** No return value; the function acts through hardware, buffers or state.	Uses: SDRAM framebuffer.
LCD_FillRectangle() main.c:268–279 Active	Clips a rectangle to the panel and fills its pixels in the active back buffer. It coordinates LCD_DrawPixel(). The implementation contains 2 loops.	**In:** uint16_t x, uint16_t y, uint16_t width, uint16_t height, uint16_t colour **Out:** No return value; the function acts through hardware, buffers or state.	Calls: LCD_DrawPixel().
LCD_ClearTextLayer() main.c:282–291 Active	Fills the complete 480×272 L8 GUI buffer with one palette entry. The implementation contains 1 loop.	**In:** uint16_t colour **Out:** No return value; the function acts through hardware, buffers or state.	Uses: SDRAM framebuffer.
LCD_LoadBackground() main.c:294–303 Active	Copies the selected weather image from memory-mapped QSPI into the SDRAM back buffer, scaling the stored half-resolution asset to the panel. The implementation contains 1 explicit decision/error check.	**In:** None. **Out:** true when the operation/result is valid; false on unavailable data, validation failure or I/O error.	Uses: SDRAM framebuffer.
LCD_RestoreBackgroundRectangle() main.c:309–333 Active	Reconstructs a damaged sub-rectangle of the back buffer from the selected QSPI background. The implementation contains 3 explicit decision/error checks and 1 loop.	**In:** uint16_t x, uint16_t y, uint16_t width, uint16_t height **Out:** No return value; the function acts through hardware, buffers or state.	Uses: SDRAM framebuffer.

Function	What it does / how it works	Interface	Dependencies
LCD_FontFromScale() main.c:336–344 Active	Maps the legacy numeric text scale to the small, medium or large proportional QSPI font. The implementation contains 2 explicit decision/error checks.	**In:** uint8_t scale **Out:** Returns QSPI_FontSize.	Pure/local helper; no direct peripheral dependency detected..
LCD_MeasureString() main.c:347–367 Active	Calculates the pixel width of a string so text can be centered or right-aligned before drawing. It coordinates LCD_FontFromScale(). The implementation contains 3 explicit decision/error checks and 1 loop.	**In:** const char *text, uint8_t scale **Out:** Returns a numeric result of type uint16_t.	Calls: LCD_FontFromScale().
LCD_DrawCharacter() main.c:374–409 Active	Renders one proportional QSPI glyph into the L8 SDRAM buffer and returns its horizontal advance. It coordinates LCD_FontFromScale(), LCD_DrawPixel(). The implementation contains 3 explicit decision/error checks and 2 loops.	**In:** uint16_t x, uint16_t y, char character, uint16_t colour, uint8_t scale **Out:** Returns a numeric result of type uint8_t.	Calls: LCD_FontFromScale(), LCD_DrawPixel().
LCD_DrawString() main.c:416–429 Active	Renders a null-terminated string from left to right using LCD_DrawCharacter. It coordinates LCD_DrawCharacter(). The implementation contains 1 explicit decision/error check and 1 loop.	**In:** uint16_t x, uint16_t y, const char *text, uint16_t colour, uint8_t scale **Out:** No return value; the function acts through hardware, buffers or state.	Calls: LCD_DrawCharacter().
LCD_DrawCenteredString() main.c:435–445 Active	Measures a string and draws it horizontally centered on the 480-pixel panel. It coordinates LCD_MeasureString(), LCD_DrawString(). The implementation contains 1 explicit decision/error check.	**In:** uint16_t y, const char *text, uint16_t colour, uint8_t scale **Out:** No return value; the function acts through hardware, buffers or state.	Calls: LCD_MeasureString(), LCD_DrawString().
LCD_MeasureStringZoom() main.c:450–469 Active	Measures a string using an explicitly selected QSPI font and integer zoom factor. The implementation contains 3 explicit decision/error checks and 1 loop.	**In:** const char *text, QSPI_FontSize font, uint8_t zoom **Out:** Returns a numeric result of type uint16_t.	Pure/local helper; no direct peripheral dependency detected..

Function	What it does / how it works	Interface	Dependencies
LCD_DrawCharacterZoom() main.c:477–523 Active	Draws one modern proportional glyph with integer pixel enlargement and clipping. It coordinates LCD_DrawPixel(). The implementation contains 3 explicit decision/error checks and 4 loops.	**In:** uint16_t x, uint16_t y, char character, uint16_t colour, QSPI_FontSize font, uint8_t zoom **Out:** Returns a numeric result of type uint16_t.	Calls: LCD_DrawPixel().
LCD_DrawStringZoom() main.c:531–549 Active	Draws a complete string with the selected modern font and zoom. It coordinates LCD_DrawCharacterZoom(). The implementation contains 2 explicit decision/error checks and 1 loop.	**In:** uint16_t x, uint16_t y, const char *text, uint16_t colour, QSPI_FontSize font, uint8_t zoom **Out:** No return value; the function acts through hardware, buffers or state.	Calls: LCD_DrawCharacterZoom().
LCD_DrawCenteredStringShadow() main.c:555–562 Active	Draws centered text twice—offset dark shadow first, coloured foreground second—for contrast on photographs. It coordinates LCD_MeasureString(), LCD_DrawString().	**In:** uint16_t y, const char *text, uint16_t colour, uint8_t scale **Out:** No return value; the function acts through hardware, buffers or state.	Calls: LCD_MeasureString(), LCD_DrawString().
LCD_DrawStringShadow() main.c:569–572 Active	Draws left-aligned shadowed text at an explicit coordinate. It coordinates LCD_DrawString().	**In:** uint16_t x, uint16_t y, const char *text, uint16_t colour, uint8_t scale **Out:** No return value; the function acts through hardware, buffers or state.	Calls: LCD_DrawString().
LCD_DrawRightStringShadow() main.c:579–584 Active	Measures and draws shadowed text against a right-hand coordinate. It coordinates LCD_MeasureString(), LCD_DrawStringShadow().	**In:** uint16_t right, uint16_t y, const char *text, uint16_t colour, uint8_t scale **Out:** No return value; the function acts through hardware, buffers or state.	Calls: LCD_MeasureString(), LCD_DrawStringShadow().

Function	What it does / how it works	Interface	Dependencies
LCD_DrawCenteredZoomShadow() main.c:591–598 Active	Centers and draws enlarged modern text with a one-pixel shadow. It coordinates LCD_MeasureStringZoom(), LCD_DrawStringZoom().	**In:** uint16_t y, const char *text, uint16_t colour, QSPI_FontSize font, uint8_t zoom **Out:** No return value; the function acts through hardware, buffers or state.	Calls: LCD_MeasureStringZoom(), LCD_DrawStringZoom().
LCD_DrawIcon() main.c:604–623 Active	Reads a 20×20 monochrome icon from QSPI and paints its set bits into the GUI buffer. It coordinates LCD_DrawPixel(). The implementation contains 2 explicit decision/error checks and 2 loops.	**In:** uint16_t x, uint16_t y, QSPI_Icon icon, uint16_t colour **Out:** No return value; the function acts through hardware, buffers or state.	Calls: LCD_DrawPixel().
LCD_PresentTextLayer() main.c:626–662 Active	Waits for vertical blanking, swaps the front/back SDRAM buffers, updates LTDC layer address, then selects the old front buffer for the next frame. It coordinates HAL_GetTick(), Error_Handler(). The implementation contains 2 explicit decision/error checks and 1 loop.	**In:** None. **Out:** No return value; the function acts through hardware, buffers or state.	Calls: __DSB(), HAL_GetTick(), Error_Handler(); Uses: LTDC, SDRAM framebuffer.
LCD_ConfigureTextLayer() main.c:665–715 Active	Programs the LTDC L8 foreground layer, loads the colour lookup table and prepares both SDRAM framebuffers for double buffering. It coordinates Error_Handler(), HAL_LTDC_ConfigLayer(), HAL_LTDC_ConfigCLUT(), HAL_LTDC_EnableCLUT(). The implementation contains 4 explicit decision/error checks.	**In:** None. **Out:** No return value; the function acts through hardware, buffers or state.	Calls: Error_Handler(), HAL_LTDC_ConfigLayer(), HAL_LTDC_ConfigCLUT(), HAL_LTDC_EnableCLUT(), __HAL_LTDC_LAYER_DISABLE(), __HAL_LTDC_LAYER_ENABLE(), __HAL_LTDC_RELOAD_CONFIG(); Uses: LTDC, SDRAM framebuffer.
LCD_TextDemo() main.c:718–737 Active	Optional bench page that exercises QSPI fonts, colours and icons before the normal dashboard starts. It coordinates LCD_ConfigureTextLayer(), LCD_ClearTextLayer(), LCD_DrawCenteredString(), LCD_PresentTextLayer().	**In:** None. **Out:** No return value; the function acts through hardware, buffers or state.	Calls: LCD_ConfigureTextLayer(), LCD_ClearTextLayer(), LCD_DrawCenteredString(), LCD_PresentTextLayer().
GUI_DrawRectangle() main.c:1229–1237 Active	Draws a rectangular outline by combining four filled edge rectangles. It coordinates LCD_FillRectangle(). The implementation contains 1 explicit decision/error check.	**In:** uint16_t x, uint16_t y, uint16_t width, uint16_t height, uint16_t colour **Out:** No return value; the function acts through hardware, buffers or state.	Calls: LCD_FillRectangle().

Function	What it does / how it works	Interface	Dependencies
GUI_DrawCard() main.c:1245–1251 Active	Paints a filled dashboard card plus its accent border. It coordinates LCD_FillRectangle(), GUI_DrawRectangle().	**In:** uint16_t x, uint16_t y, uint16_t width, uint16_t height, uint16_t fill, uint16_t accent **Out:** No return value; the function acts through hardware, buffers or state.	Calls: LCD_FillRectangle(), GUI_DrawRectangle().
GUI_DrawCenteredInBox() main.c:1259–1267 Active	Centers a text string inside an arbitrary horizontal box. It coordinates LCD_MeasureString(), LCD_DrawString(). The implementation contains 1 explicit decision/error check.	**In:** uint16_t x, uint16_t width, uint16_t y, const char *text, uint16_t colour, uint8_t scale **Out:** No return value; the function acts through hardware, buffers or state.	Calls: LCD_MeasureString(), LCD_DrawString().
GUI_FormatSignedTenths() main.c:1273–1288 Active	Formats a fixed-point signed value stored in tenths without requiring floating-point arithmetic. The implementation contains 1 explicit decision/error check.	**In:** char *buffer, size_t capacity, int32_t value10, const char *unit **Out:** No return value; the function acts through hardware, buffers or state.	Pure/local helper; no direct peripheral dependency detected..
GUI_ShowDashboard() main.c:1503–1722 Active	Legacy indoor-only dashboard renderer retained as a fallback/test view. It coordinates AirQuality_Get(), AirQuality_Colour(), LCD_LoadBackground(), Error_Handler(), LCD_DrawString(), GUI_DrawCard(). The implementation contains 10 explicit decision/error checks.	**In:** bool scd_ok, bool bme_ok, bool wifi_ok, bool blynk_ok, const SensorValues *values **Out:** No return value; the function acts through hardware, buffers or state.	Calls: AirQuality_Get(), AirQuality_Colour(), LCD_LoadBackground(), Error_Handler(), LCD_DrawString(), GUI_DrawCard(), LCD_DrawIcon(), LCD_RestoreBackgroundRectangle().
GUI_ShowAlarm() main.c:1727–1913 Active	Renders the highest-priority alarm page and updates only the changing value area when possible. It coordinates GUI_FormatSignedTenths(), LCD_ClearTextLayer(), GUI_DrawRectangle(), GUI_DrawCenteredInBox(), LCD_FillRectangle(), LCD_PresentTextLayer(). The implementation contains 4 explicit decision/error checks.	**In:** SystemAlarm alarm, bool flash, const SensorValues *values **Out:** No return value; the function acts through hardware, buffers or state.	Calls: GUI_FormatSignedTenths(), LCD_ClearTextLayer(), GUI_DrawRectangle(), GUI_DrawCenteredInBox(), LCD_FillRectangle(), LCD_PresentTextLayer().

Function	What it does / how it works	Interface	Dependencies
LCD_ShowStatus() main.c:2376–2386 Active	Shows a two-line startup status page, but deliberately becomes a no-op after the live dashboard is active. It coordinates LCD_ClearTextLayer(), LCD_DrawCenteredString(), LCD_PresentTextLayer(). The implementation contains 1 explicit decision/error check.	**In:** const char *line1, const char *line2, uint16_t colour **Out:** No return value; the function acts through hardware, buffers or state.	Calls: LCD_ClearTextLayer(), LCD_DrawCenteredString(), LCD_PresentTextLayer().
GUI_ShowWeatherDashboard() main.c:4167–4363 Active	Builds the normal weather-and-room dashboard from cached sensor, location, weather and network state, then presents one complete frame. It coordinates Clock_Format(), LCD_LoadBackground(), Error_Handler(), LCD_MeasureString(), HAL_GetTick(), LCD_DrawStringShadow(). The implementation contains 5 explicit decision/error checks.	**In:** bool scd_ok, bool bme_ok, bool wifi_ok, bool blynk_ok, const SensorValues *values, const InternetWeather *weather **Out:** No return value; the function acts through hardware, buffers or state.	Calls: Clock_Format(), LCD_LoadBackground(), Error_Handler(), LCD_MeasureString(), HAL_GetTick(), LCD_DrawStringShadow(), LCD_DrawRightStringShadow(), LCD_DrawCenteredStringShadow().

2.5 Environmental sensors

11 functions in this subsystem.

Function	What it does / how it works	Interface	Dependencies
SCD41_CRC8() main.c:826–843 Active	Computes the Sensirion CRC-8 used to validate each two-byte SCD41 data word. The implementation contains 1 explicit decision/error check and 2 loops.	**In:** const uint8_t *data **Out:** Returns a numeric result of type uint8_t.	Pure/local helper; no direct peripheral dependency detected..
SCD41_SendCommand() main.c:846–857 Active	Serializes a 16-bit command in big-endian order and transmits it on I2C4. It coordinates HAL_I2C_Master_Transmit().	**In:** uint16_t command **Out:** HAL_OK on success; HAL_ERROR (or another HAL status) is propagated on failure.	Calls: HAL_I2C_Master_Transmit(); Uses: I2C4/SCD41.
SCD41_IsDataReady() main.c:860–884 Active	Requests the SCD41 data-ready status, checks its CRC and reports whether a fresh sample is available. It coordinates SCD41_SendCommand(), HAL_Delay(), HAL_I2C_Master_Receive(), SCD41_CRC8(). The implementation contains 3 explicit decision/error checks.	**In:** None. **Out:** true when the operation/result is valid; false on unavailable data, validation failure or I/O error.	Calls: SCD41_SendCommand(), HAL_Delay(), HAL_I2C_Master_Receive(), SCD41_CRC8(); Uses: I2C4/SCD41.

Function	What it does / how it works	Interface	Dependencies
SCD41_Init() main.c:887–910 Active	Checks the SCD41 address, stops any previous measurement session, then starts periodic CO_2 measurement. It coordinates HAL_I2C_IsDeviceReady(), SCD41_SendCommand(), HAL_Delay(). The implementation contains 2 explicit decision/error checks.	**In:** None. **Out:** true when the operation/result is valid; false on unavailable data, validation failure or I/O error.	Calls: HAL_I2C_IsDeviceReady(), SCD41_SendCommand(), HAL_Delay(); Uses: I2C4/SCD41.
SCD41_Read() main.c:913–972 Active	Reads the SCD41 CO_2, temperature and humidity words, verifies all CRC bytes and updates SensorValues. It coordinates SCD41_IsDataReady(), HAL_Delay(), SCD41_SendCommand(), HAL_I2C_Master_Receive(), SCD41_CRC8(). The implementation contains 5 explicit decision/error checks and 1 loop.	**In:** SensorValues *values **Out:** true when the operation/result is valid; false on unavailable data, validation failure or I/O error.	Calls: SCD41_IsDataReady(), HAL_Delay(), SCD41_SendCommand(), HAL_I2C_Master_Receive(), SCD41_CRC8(); Uses: I2C4/SCD41.
BME688_I2C_Read() main.c:993–1006 Active	Bosch-driver callback that reads BME688 registers through STM32 HAL I2C3. It coordinates HAL_I2C_Mem_Read().	**In:** uint8_t reg, uint8_t *data, uint32_t length, void *interface_ptr **Out:** Returns a numeric result of type int8_t.	Calls: HAL_I2C_Mem_Read().
BME688_I2C_Write() main.c:1012–1025 Active	Bosch-driver callback that writes BME688 registers through STM32 HAL I2C3. It coordinates HAL_I2C_Mem_Write().	**In:** uint8_t reg, const uint8_t *data, uint32_t length, void *interface_ptr **Out:** Returns a numeric result of type int8_t.	Calls: HAL_I2C_Mem_Write().
BME688_DelayUs() main.c:1028–1033 Active	Bosch-driver delay callback implemented with HAL_Delay at millisecond resolution. It coordinates HAL_Delay(). The implementation contains 1 explicit decision/error check.	**In:** uint32_t period_us, void *interface_ptr **Out:** No return value; the function acts through hardware, buffers or state.	Calls: HAL_Delay().
BME688_Init() main.c:1036–1095 Active	Connects the Bosch BME68x API to I2C3, verifies the chip and configures oversampling, filter and heater settings. It coordinates HAL_I2C_IsDeviceReady(). The implementation contains 5 explicit decision/error checks.	**In:** None. **Out:** true when the operation/result is valid; false on unavailable data, validation failure or I/O error.	Calls: HAL_I2C_IsDeviceReady(); Uses: I2C3/BME688.

Function	What it does / how it works	Interface	Dependencies
BME688_Read() main.c:1098–1163 Active	Triggers forced mode, waits for conversion, obtains compensated temperature/humidity/pressure/gas values and stores fixed-point results. The implementation contains 4 explicit decision/error checks.	**In:** SensorValues *values **Out:** true when the operation/result is valid; false on unavailable data, validation failure or I/O error.	Pure/local helper; no direct peripheral dependency detected..
BME688_GasTestScreen() main.c:2694–2859 Active	Standalone gas commissioning page that shows raw gas resistance, baseline progress, ratio and alarm state. It coordinates LCD_ClearTextLayer(), LCD_DrawString(), LCD_PresentTextLayer(), BME688_Read(). The implementation contains 9 explicit decision/error checks.	**In:** bool sensor_online **Out:** No return value; the function acts through hardware, buffers or state.	Calls: LCD_ClearTextLayer(), LCD_DrawString(), LCD_PresentTextLayer(), BME688_Read().

2.6 Flame and gas safety

5 functions in this subsystem.

Function	What it does / how it works	Interface	Dependencies
FlameSensor_ReadADC() main.c:2448–2470 Active	Starts ADC1 on PA1, polls for conversion and returns the raw flame phototransistor value. It coordinates HAL_ADC_Start(), HAL_ADC_PollForConversion(), HAL_ADC_Stop(), HAL_ADC_GetValue(). The implementation contains 3 explicit decision/error checks.	**In:** uint16_t *raw **Out:** HAL_OK on success; HAL_ERROR (or another HAL status) is propagated on failure.	Calls: HAL_ADC_Start(), HAL_ADC_PollForConversion(), HAL_ADC_Stop(), HAL_ADC_GetValue(); Uses: ADC1/PA1.
FlameSensor_ReadAll() main.c:2473–2489 Active	Combines the PA1 analog reading with PH2 digital comparator input and updates flame voltage/detected fields. It coordinates HAL_GPIO_ReadPin(), FlameSensor_ReadADC(). The implementation contains 2 explicit decision/error checks.	**In:** SensorValues *values **Out:** true when the operation/result is valid; false on unavailable data, validation failure or I/O error.	Calls: HAL_GPIO_ReadPin(), FlameSensor_ReadADC().
GasMonitor_Update() main.c:2492–2600 Active	Learns a clean-air BME688 gas-resistance baseline, calculates the current percentage ratio and applies confirm/clear counters for stable alarms. The implementation contains 12 explicit decision/error checks.	**In:** SensorValues *values **Out:** No return value; the function acts through hardware, buffers or state.	Pure/local helper; no direct peripheral dependency detected..
FlameSensor_TestScreen() main.c:2604–2692 Active	Standalone flame-sensor test loop that displays ADC, voltage and digital state. It coordinates HAL_GPIO_ReadPin(), LCD_ClearTextLayer(), LCD_DrawString(), FlameSensor_ReadADC(), HAL_UART_Transmit(), LCD_PresentTextLayer(). The implementation contains 2 explicit decision/error checks.	**In:** None. **Out:** No return value; the function acts through hardware, buffers or state.	Calls: HAL_GPIO_ReadPin(), LCD_ClearTextLayer(), LCD_DrawString(), FlameSensor_ReadADC(), HAL_UART_Transmit(), LCD_PresentTextLayer(); Uses: USART3 debug.

Function	What it does / how it works	Interface	Dependencies
Safety_FlameActiveDuringNetwork() main.c:2867–2881 Active	Samples the flame digital input during blocking ESP operations so a fire condition can abort network work immediately. It coordinates HAL_GPIO_ReadPin(), HAL_GPIO_WritePin(). The implementation contains 1 explicit decision/error check.	In: None. Out: true when the operation/result is valid; false on unavailable data, validation failure or I/O error.	Calls: HAL_GPIO_ReadPin(), HAL_GPIO_WritePin(); Uses: buzzer GPIO.

2.7 Air-quality and alarm logic

6 functions in this subsystem.

Function	What it does / how it works	Interface	Dependencies
AirQuality_Get() main.c:1293–1311 Active	Classifies indoor air as unknown, good, fair, poor or danger using sensor health, CO_2 and learned gas-ratio thresholds. The implementation contains 4 explicit decision/error checks.	In: bool scd_ok, bool bme_ok, const SensorValues *values Out: AirQualityLevel classification enum.	Pure/local helper; no direct peripheral dependency detected..
AirQuality_Name() main.c:1314–1323 Active	Converts an AirQualityLevel enum into the short label shown on the display.	In: AirQualityLevel level Out: Pointer to a static/received text value; ownership is not transferred.	Pure/local helper; no direct peripheral dependency detected..
AirQuality_Explanation() main.c:1327–1354 Active	Selects the user-facing ventilation or sensor-status explanation for the current air-quality state. The implementation contains 1 explicit decision/error check.	In: AirQualityLevel level, const SensorValues *values Out: Pointer to a static/received text value; ownership is not transferred.	Pure/local helper; no direct peripheral dependency detected..
AirQuality_Colour() main.c:1357–1366 Active	Maps each air-quality state to its palette colour.	In: AirQualityLevel level Out: Returns a numeric result of type uint16_t.	Pure/local helper; no direct peripheral dependency detected..

Function	What it does / how it works	Interface	Dependencies
SystemAlarm_Evaluate() main.c:1373–1465 Active	Applies alarm priority and thresholds to flame, sensor-health, CO_2, temperature, humidity, pressure and gas state, returning an alarm type plus buzzer pattern. The implementation contains 13 explicit decision/error checks.	**In:** bool measurements_ready, bool scd_healthy, bool bme_healthy, bool flame_sensor_healthy, const SensorValues *values **Out:** SystemAlarm containing the selected alarm type and buzzer pattern.	Uses: buzzer GPIO.
Buzzer_Update() main.c:1468–1496 Active	Runs the non-blocking buzzer state machine and drives the GPIO according to off, warning or critical cadence. It coordinates HAL_GPIO_WritePin().	**In:** BuzzerPattern pattern, uint32_t now **Out:** No return value; the function acts through hardware, buffers or state.	Calls: HAL_GPIO_WritePin(); Uses: buzzer GPIO.

2.8 ESP-01 and HTTP transport

9 functions in this subsystem.

Function	What it does / how it works	Interface	Dependencies
ESP01_SendCommand() main.c:2887–2939 Active	Flushes stale UART bytes, sends an AT command to USART1 and collects the response until timeout while monitoring flame safety. It coordinates HAL_UART_Receive(), HAL_UART_Transmit(), HAL_GetTick(), Safety_FlameActiveDuringNetwork(). The implementation contains 5 explicit decision/error checks and 2 loops.	**In:** const char *command, char *response, uint16_t capacity, uint32_t timeout_ms **Out:** Returns a numeric result of type uint16_t.	Calls: HAL_UART_Receive(), HAL_UART_Transmit(), HAL_GetTick(), Safety_FlameActiveDuringNetwork(); Uses: USART1/ESP-01.
ESP01_TestScreen() main.c:2941–3015 Active	Resets/probes the ESP-01 and confirms that its AT firmware responds before Wi-Fi setup proceeds. It coordinates HAL_Delay(), HAL_UART_Init(), ESP01_SendCommand(), LCD_ClearTextLayer(), LCD_DrawString(), HAL_UART_Transmit(). The implementation contains 3 explicit decision/error checks and 1 loop.	**In:** None. **Out:** true when the operation/result is valid; false on unavailable data, validation failure or I/O error.	Calls: HAL_Delay(), HAL_UART_Init(), ESP01_SendCommand(), LCD_ClearTextLayer(), LCD_DrawString(), HAL_UART_Transmit(), LCD_PresentTextLayer(); Uses: USART1/ESP-01, USART3 debug.
ESP01_ConnectWiFi() main.c:3017–3167 Active	Configures station mode, joins the configured access point and verifies that the module obtained an IP connection. It coordinates ESP01_SendCommand(), LCD_ShowStatus(), HAL_UART_Transmit(), LCD_ClearTextLayer(), LCD_DrawString(), LCD_PresentTextLayer(). The implementation contains 6 explicit decision/error checks.	**In:** None. **Out:** true when the operation/result is valid; false on unavailable data, validation failure or I/O error.	Calls: ESP01_SendCommand(), LCD_ShowStatus(), HAL_UART_Transmit(), LCD_ClearTextLayer(), LCD_DrawString(), LCD_PresentTextLayer(); Uses: USART3 debug.

Function	What it does / how it works	Interface	Dependencies
ESP01_ReadUntil() main.c:3173–3212 Active	Receives USART1 bytes into a bounded buffer until success text, failure text, safety abort or timeout occurs. It coordinates HAL_GetTick(), Safety_FlameActiveDuringNetwork(), HAL_UART_Receive(). The implementation contains 5 explicit decision/error checks and 1 loop.	**In:** char *response, uint16_t capacity, const char *success, const char *failure, uint32_t timeout_ms **Out:** Returns a numeric result of type uint16_t.	Calls: HAL_GetTick(), Safety_FlameActiveDuringNetwork(), HAL_UART_Receive(); Uses: USART1/ESP-01.
ESP01_PrepareDataSend() main.c:3215–3270 Active	Opens the ESP AT send prompt for an exact HTTP payload length and verifies the prompt before transmission. It coordinates HAL_UART_Receive(), HAL_UART_Transmit(), ESP01_ReadUntil(). The implementation contains 2 explicit decision/error checks and 1 loop.	**In:** uint16_t data_length **Out:** true when the operation/result is valid; false on unavailable data, validation failure or I/O error.	Calls: HAL_UART_Receive(), HAL_UART_Transmit(), ESP01_ReadUntil(); Uses: USART1/ESP-01, USART3 debug.
Text_ContainsInsensitive() main.c:3278–3300 Active	Performs a case-insensitive substring test without changing either input string. The implementation contains 2 explicit decision/error checks and 2 loops.	**In:** const char *text, const char *word **Out:** true when the requested text is found; otherwise false.	Pure/local helper; no direct peripheral dependency detected..
Text_ToDisplayUpper() main.c:3305–3344 Active	Copies text into a bounded display buffer while converting letters to uppercase and replacing unsupported characters. The implementation contains 5 explicit decision/error checks and 2 loops.	**In:** char *destination, size_t capacity, const char *source **Out:** No return value; the function acts through hardware, buffers or state.	Pure/local helper; no direct peripheral dependency detected..
ESP01_HTTPGet() main.c:3351–3464 Active	Opens a TCP socket, sends one HTTP/1.1 GET request, captures the modem/HTTP response and accepts only a successful status. It coordinates ESP01_SendCommand(), HAL_Delay(), ESP01_PrepareDataSend(), HAL_UART_Transmit(), ESP01_ReadUntil(). The implementation contains 7 explicit decision/error checks.	**In:** const char *host, const char *path, char *response, uint16_t response_capacity, uint32_t timeout_ms **Out:** true when the operation/result is valid; false on unavailable data, validation failure or I/O error.	Calls: ESP01_SendCommand(), HAL_Delay(), ESP01_PrepareDataSend(), HAL_UART_Transmit(), ESP01_ReadUntil(); Uses: USART1/ESP-01, USART3 debug.

Function	What it does / how it works	Interface	Dependencies
HTTP_Body() main.c:3467–3488 Active	Returns the first byte after the HTTP header separator, or the original pointer when no separator exists. The implementation contains 4 explicit decision/error checks.	**In:** const char *response **Out:** Pointer to a static/received text value; ownership is not transferred.	Pure/local helper; no direct peripheral dependency detected..

2.9 Location, time and weather

16 functions in this subsystem.

Function	What it does / how it works	Interface	Dependencies
Date_MonthNumber() main.c:3491–3505 Active	Converts a three-letter HTTP month name into month number 1–12. The implementation contains 1 explicit decision/error check and 1 loop.	**In:** const char *month **Out:** Returns a numeric result of type int32_t.	Pure/local helper; no direct peripheral dependency detected..
Date_DaysFromCivil() main.c:3510–3522 Active	Converts a Gregorian date to a day count relative to the Unix epoch using an integer civil-date algorithm.	**In:** int32_t year, uint32_t month, uint32_t day **Out:** Returns a numeric result of type int64_t.	Pure/local helper; no direct peripheral dependency detected..
Clock_SyncFromHTTPDate() main.c:3526–3579 Active	Parses an HTTP Date header, converts it to UTC epoch seconds and stores the local clock synchronization reference. It coordinates Date_MonthNumber(), Date_DaysFromCivil(), HAL_GetTick(). The implementation contains 5 explicit decision/error checks.	**In:** InternetWeather *weather, const char *response **Out:** true when the operation/result is valid; false on unavailable data, validation failure or I/O error.	Calls: Date_MonthNumber(), Date_DaysFromCivil(), HAL_GetTick().
Clock_LocalEpoch() main.c:3582–3597 Active	Advances the last synchronized UTC epoch by HAL tick elapsed time and applies the geolocated UTC offset. It coordinates HAL_GetTick(). The implementation contains 2 explicit decision/error checks.	**In:** const InternetWeather *weather **Out:** Returns a numeric result of type uint32_t.	Calls: HAL_GetTick().
Clock_CivilFromDays() main.c:3603–3625 Active	Converts an epoch day count back into Gregorian year, month and day.	**In:** int64_t days, int32_t *year, uint32_t *month, uint32_t *day **Out:** No return value; the function acts through hardware, buffers or state.	Pure/local helper; no direct peripheral dependency detected..

Function	What it does / how it works	Interface	Dependencies
Clock_Format() main.c:3633–3678 Active	Produces display-ready local time/date strings and optionally returns the current minute of day. It coordinates Clock_LocalEpoch(), Clock_CivilFromDays(). The implementation contains 5 explicit decision/error checks.	In: const InternetWeather *weather, char *time_text, size_t time_capacity, char *date_text, size_t date_capacity, uint16_t *minutes_today Out: No return value; the function acts through hardware, buffers or state.	Calls: Clock_LocalEpoch(), Clock_CivilFromDays().
JSON_CopyValue() main.c:3684–3729 Active	Copies a simple quoted or scalar JSON value associated with a key into a bounded destination buffer. The implementation contains 5 explicit decision/error checks and 2 loops.	In: const char *json, const char *key, char *destination, size_t capacity Out: true when the operation/result is valid; false on unavailable data, validation failure or I/O error.	Pure/local helper; no direct peripheral dependency detected..
Location_Fetch() main.c:3732–3789 Active	Queries ip-api.com, parses city/coordinates/time-zone offset and synchronizes the clock from the HTTP response date. It coordinates ESP01_HTTPGet(), HTTP_Body(), JSON_CopyValue(), Text_ToDisplayUpper(), Clock_SyncFromHTTPDate(). The implementation contains 4 explicit decision/error checks.	In: InternetWeather *weather Out: true when the operation/result is valid; false on unavailable data, validation failure or I/O error.	Calls: ESP01_HTTPGet(), HTTP_Body(), JSON_CopyValue(), Text_ToDisplayUpper(), Clock_SyncFromHTTPDate().
Weather_JSONValueStart() main.c:3793–3824 Active	Finds the start of a named JSON value within a selected Open-Meteo response section. The implementation contains 3 explicit decision/error checks and 2 loops.	In: const char *section, const char *key Out: Pointer to a static/received text value; ownership is not transferred.	Pure/local helper; no direct peripheral dependency detected..
Weather_JSONRoundedInteger() main.c:3830–3855 Active	Parses one JSON number and rounds it to a signed 16-bit integer. It coordinates Weather_JSONValueStart(). The implementation contains 4 explicit decision/error checks.	In: const char *section, const char *key, int16_t *result Out: true when the operation/result is valid; false on unavailable data, validation failure or I/O error.	Calls: Weather_JSONValueStart().

Function	What it does / how it works	Interface	Dependencies
Weather_JSONFirstString() main.c:3861–3884 Active	Copies the first string element of a JSON array into a bounded destination. It coordinates Weather_JSONValueStart(). The implementation contains 4 explicit decision/error checks.	**In:** const char *section, const char *key, char *destination, size_t capacity **Out:** true when the operation/result is valid; false on unavailable data, validation failure or I/O error.	Calls: Weather_JSONValueStart().
Weather_ParseMinutes() main.c:3887–3909 Active	Converts an ISO time ending in HH:MM into minutes after midnight. It coordinates Text_ContainsInsensitive(). The implementation contains 4 explicit decision/error checks.	**In:** const char *text **Out:** Returns a numeric result of type uint16_t.	Calls: Text_ContainsInsensitive().
Weather_SelectTheme() main.c:3912–4000 Active	Chooses clear-day, dawn, sunset, night, cloudy, rain, storm or snow background based on weather code and local time. It coordinates Text_ContainsInsensitive(), Clock_Format(). The implementation contains 11 explicit decision/error checks.	**In:** InternetWeather *weather **Out:** No return value; the function acts through hardware, buffers or state.	Calls: Text_ContainsInsensitive(), Clock_Format().
Weather_WMODescription() main.c:4003–4028 Active	Maps a WMO weather code and day/night flag to a short human-readable condition. The implementation contains 11 explicit decision/error checks.	**In:** uint16_t code, bool is_day **Out:** Pointer to a static/received text value; ownership is not transferred.	Pure/local helper; no direct peripheral dependency detected..
Weather_ParseOpenMeteo() main.c:4032–4120 Active	Extracts current temperature, apparent temperature, min/max, weather code, sunrise/sunset and update time from Open-Meteo JSON. It coordinates Weather_JSONRoundedInteger(), Weather_JSONFirstString(), Weather_WMODescription(), Weather_ParseMinutes(), HAL_GetTick(), Weather_SelectTheme(). The implementation contains 5 explicit decision/error checks.	**In:** InternetWeather *weather, const char *body **Out:** true when the operation/result is valid; false on unavailable data, validation failure or I/O error.	Calls: Weather_JSONRoundedInteger(), Weather_JSONFirstString(), Weather_WMODescription(), Weather_ParseMinutes(), HAL_GetTick(), Weather_SelectTheme().
Weather_Fetch() main.c:4123–4159 Active	Builds the Open-Meteo request from detected coordinates, performs HTTP GET, parses the response and updates validity/theme timestamps. It coordinates ESP01_HTTPGet(), Clock_SyncFromHTTPDate(), HTTP_Body(), Weather_ParseOpenMeteo(). The implementation contains 3 explicit decision/error checks.	**In:** InternetWeather *weather **Out:** true when the operation/result is valid; false on unavailable data, validation failure or I/O error.	Calls: ESP01_HTTPGet(), Clock_SyncFromHTTPDate(), HTTP_Body(), Weather_ParseOpenMeteo().

2.10 Blynk telemetry and formatting

5 functions in this subsystem.

Function	What it does / how it works	Interface	Dependencies
Blynk_UpdateVirtualPin() main.c:4367–4702 Active	Sends one value to one Blynk virtual pin through a plain-HTTP API request and validates the HTTP response. It coordinates LCD_ShowStatus(), HAL_Delay(), ESP01_SendCommand(), HAL_UART_Transmit(), ESP01_PrepareDataSend(), ESP01_ReadUntil(). The implementation contains 15 explicit decision/error checks.	**In:** uint8_t virtual_pin, const char *value **Out:** true when the operation/result is valid; false on unavailable data, validation failure or I/O error.	Calls: LCD_ShowStatus(), HAL_Delay(), ESP01_SendCommand(), HAL_UART_Transmit(), ESP01_PrepareDataSend(), ESP01_ReadUntil(); Uses: USART1/ESP-01, USART3 debug.
BufferAppend() main.c:4708–4735 Active	Bounds-checks and appends formatted text to a shared request buffer using vsnprintf. The implementation contains 2 explicit decision/error checks.	**In:** char *buffer, size_t capacity, size_t *used, const char *format, ... **Out:** true when the operation/result is valid; false on unavailable data, validation failure or I/O error.	Pure/local helper; no direct peripheral dependency detected..
FormatSignedTenthsValue() main.c:4740–4757 Active	Converts a signed fixed-point tenths value into a compact decimal string. The implementation contains 2 explicit decision/error checks.	**In:** char *buffer, size_t capacity, int32_t value10 **Out:** No return value; the function acts through hardware, buffers or state.	Pure/local helper; no direct peripheral dependency detected..
FormatUnsignedTenthsValue() main.c:4762–4771 Active	Converts an unsigned fixed-point tenths value into a compact decimal string. The implementation contains 1 explicit decision/error check.	**In:** char *buffer, size_t capacity, uint32_t value10 **Out:** No return value; the function acts through hardware, buffers or state.	Pure/local helper; no direct peripheral dependency detected..
Blynk_UpdateAllMeasurements() main.c:4778–5028 Active	Formats all current readings and transmits them in one batched Blynk HTTP update to reduce socket overhead. It coordinates BufferAppend(), FormatSignedTenthsValue(), FormatUnsignedTenthsValue(), ESP01_SendCommand(), HAL_Delay(), ESP01_PrepareDataSend(). The implementation contains 20 explicit decision/error checks.	**In:** bool scd_ok, bool bme_ok, bool flame_adc_ok, const SensorValues *values, const InternetWeather *weather **Out:** true when the operation/result is valid; false on unavailable data, validation failure or I/O error.	Calls: BufferAppend(), FormatSignedTenthsValue(), FormatUnsignedTenthsValue(), ESP01_SendCommand(), HAL_Delay(), ESP01_PrepareDataSend(), HAL_UART_Transmit(), ESP01_ReadUntil(); Uses: USART1/ESP-01, USART3 debug.

3 Critical-function walkthroughs

The execution paths that are most important for debugging and future modification

1. main() — Cooperative application scheduler

```
int main(void)
```

Initializes memory, peripherals, sensors and Wi-Fi, then runs the cooperative scheduler for sampling, alarms, GUI, weather and Blynk uploads.

1. Configures MPU and clocks, initializes all CubeMX peripherals, starts SDRAM, validates/maps QSPI assets, and only then enables the LTDC GUI.
2. Initializes SCD41, BME688 and ESP-01; when Wi-Fi is available it performs location detection followed by the first weather request.
3. Samples flame every 200 ms and environmental sensors every 5 s, maintaining health counters and retrying disconnected I²C sensors.
4. Evaluates alarms and services the buzzer before any network work, refreshes the display every 500 ms, and serializes location/weather/Blynk requests so only one blocking request happens at a time.
5. Uses HAL_GetTick subtraction for wrap-safe timing and finishes each loop with a short 10 ms delay.

IMPLEMENTATION NOTE

The design is intentionally cooperative rather than RTOS-based. Any new network operation must preserve flame checks and must not add long unmonitored blocking delays.

2. SDRAM_Startup() — IS42S16400J power-up sequence

```
static HAL_StatusTypeDef SDRAM_Startup(void)
```

Sends the JEDEC SDRAM power-up command sequence and programs the proven 66.7 MHz refresh count.

1. Send CLOCK ENABLE to SDRAM Bank 1.
2. Wait at least 100 µs (implemented conservatively as 1 ms).
3. Precharge all banks, then issue eight auto-refresh cycles.
4. Load burst-length 1, sequential burst, CAS 3 and single-write-burst mode.
5. Program refresh count 1022 for FMC SDCLK ≈66.7 MHz and 4096 rows.

IMPLEMENTATION NOTE

Nothing may access 0xC0000000 before this function returns HAL_OK.

3. QSPI_Assets_Init() — Persistent asset-store boot path

```
HAL_StatusTypeDef QSPI_Assets_Init(QSPI_HandleTypeDef *hqspi, bool *installed_now)
```

Resets and identifies the flash, enables quad mode, installs assets when requested, validates CRCs and enters memory-mapped mode.

1. Validate the HAL handle, reset the W25Q128 and read JEDEC ID EF 40 18.

2. Enable quad mode and read the versioned asset header.
3. When the installer build is enabled and sources differ, erase/program/verify the asset region once.
4. Validate header geometry and CRCs, then enter memory-mapped quad-read mode at 0x90000000.

IMPLEMENTATION NOTE

After a successful one-time installation, QSPI_ASSET_INSTALLER should be set to 0 and the large generated source arrays excluded from normal firmware builds.

4. LCD_ConfigureTextLayer() — Double-buffer creation

```
static void LCD_ConfigureTextLayer(void)
```

Programs the LTDC L8 foreground layer, loads the colour lookup table and prepares both SDRAM framebuffers for double buffering.

1. Assign two 480×272 L8 frames at 0xC0000000 and 0xC0040000.
2. Clear both frames and load the 256-entry palette from memory-mapped QSPI.
3. Configure LTDC foreground layer format, window and first frame address.
4. Enable double-buffer state so subsequent drawing always targets the non-visible frame.

IMPLEMENTATION NOTE

The SDRAM MPU region is non-cacheable, so CPU writes are immediately visible to LTDC without cache maintenance.

5. LCD_PresentTextLayer() — Tear-free frame presentation

```
static void LCD_PresentTextLayer(void)
```

Waits for vertical blanking, swaps the front/back SDRAM buffers, updates LTDC layer address, then selects the old front buffer for the next frame.

1. Request LTDC vertical-blank reload and wait for the safe display interval.
2. Swap the software front/back pointers.
3. Program the LTDC layer framebuffer address with the completed back frame.
4. Reload the layer and redirect all later drawing to the old front frame.

IMPLEMENTATION NOTE

All screen composition should finish before this call; presenting repeatedly during drawing would reintroduce flashing or tearing.

6. GUI_ShowWeatherDashboard() — Normal dashboard composition

```
static void GUI_ShowWeatherDashboard(bool scd_ok, bool bme_ok, bool wifi_ok, bool blynk_ok, const SensorValues *values, const InternetWeather *weather)
```

Builds the normal weather-and-room dashboard from cached sensor, location, weather and network state, then presents one complete frame.

1. Reject null data, format local time/date, and compare current state with cached last state to skip identical frames.
2. Copy the weather-dependent background from QSPI to the SDRAM back frame.

3. Draw location/outside weather and the room temperature, humidity, CO_2, pressure, air-quality explanation and network state using QSPI fonts/icons.
4. Present once, then cache every displayed input so the next call only rebuilds when something changed.

IMPLEMENTATION NOTE

This change-detection plus one-shot buffer swap is the reason live values update without the whole panel visibly flashing.

7. SCD41_Read() — CO_2 measurement transaction

```
static bool SCD41_Read(SensorValues *values)
```

Reads the SCD41 CO_2, temperature and humidity words, verifies all CRC bytes and updates SensorValues.

1. Check the data-ready command first; return without corrupting prior values if no fresh sample exists.
2. Issue Read Measurement and receive three two-byte words, each followed by its own CRC byte.
3. Reject the complete sample if any CRC fails.
4. Convert raw temperature and humidity with Sensirion formulas and store CO_2 plus tenths-scaled values.

IMPLEMENTATION NOTE

A false return means the caller should keep the last known sample and increment its health counter.

8. BME688_Read() — Forced-mode environmental conversion

```
static bool BME688_Read(SensorValues *values)
```

Triggers forced mode, waits for conversion, obtains compensated temperature/humidity/pressure/gas values and stores fixed-point results.

1. Set forced mode so one conversion uses the configured oversampling and heater profile.
2. Calculate and wait for the required measurement duration.
3. Read Bosch-compensated field data and accept only a new valid measurement.
4. Convert temperature, humidity, pressure and gas resistance into the integer/fixed-point representation used by the UI and Blynk.

IMPLEMENTATION NOTE

The Bosch BME68x driver performs compensation; this function owns scheduling and project-specific unit conversion.

9. GasMonitor_Update() — Adaptive gas-change detector

```
static void GasMonitor_Update(SensorValues *values)
```

Learns a clean-air BME688 gas-resistance baseline, calculates the current percentage ratio and applies confirm/clear counters for stable alarms.

1. Accumulate a clean-air resistance average during the warm-up/baseline window.
2. Express each new gas resistance as a percentage of the baseline.
3. Require several consecutive low-ratio samples before asserting the gas alarm.
4. Require several consecutive recovered samples before clearing it, providing hysteresis and noise rejection.

IMPLEMENTATION NOTE

The gas state is monitored in the background and used for alarms/Blynk; it is intentionally not presented as an absolute gas concentration.

10. SystemAlarm_Evaluate() — Alarm priority encoder

```
static SystemAlarm SystemAlarm_Evaluate(bool measurements_ready, bool scd_healthy, bool bme_healthy, bool flame_sensor_healthy, const SensorValues *values)
```

Applies alarm priority and thresholds to flame, sensor-health, CO_2, temperature, humidity, pressure and gas state, returning an alarm type plus buzzer pattern.

1. Return no alarm until the first scheduled measurement pass is complete.
2. Evaluate flame first, then sensor failures and environmental danger conditions in fixed priority order.
3. Associate each alarm with either critical or warning buzzer cadence.
4. Return the first/highest-priority condition so only one clear instruction is displayed.

IMPLEMENTATION NOTE

Adding a new hazard requires an explicit priority decision; do not simply append it without considering flame and sensor-failure behavior.

11. ESP01_HTTPGet() — Reusable plain-HTTP GET transaction

```
static bool ESP01_HTTPGet(const char *host, const char *path, char *response, uint16_t response_capacity, uint32_t timeout_ms)
```

Opens a TCP socket, sends one HTTP/1.1 GET request, captures the modem/HTTP response and accepts only a successful status.

1. Close any stale socket and open TCP to the requested host on port 80.
2. Build a bounded HTTP/1.1 request with Host and Connection: close headers.
3. Ask the ESP for an exact send length, transmit immediately after the prompt, then collect the response until CLOSED/ERROR/timeout.
4. Mirror diagnostics to USART3 and accept only HTTP 200.

IMPLEMENTATION NOTE

The current implementation is HTTP, not TLS. Authentication tokens therefore should only be used on a trusted network for this student prototype.

12. Weather_Fetch() — Outside-weather update

```
static bool Weather_Fetch(InternetWeather *weather)
```

Builds the Open-Meteo request from detected coordinates, performs HTTP GET, parses the response and updates validity/theme timestamps.

1. Require valid geolocation coordinates and build an Open-Meteo path for current/daily values in the detected timezone.
2. Execute ESP01_HTTPGet and synchronize the local clock from the response Date header.

3. Pass the HTTP body to Weather_ParseOpenMeteo; preserve old valid weather if the new response cannot be parsed.
4. On success update validity/timestamp and select the appropriate QSPI theme.

IMPLEMENTATION NOTE

The normal scheduler retries failures after one minute and refreshes successful weather every fifteen minutes.

13. Blynk_UpdateAllMeasurements() — Batched cloud upload

```
static bool Blynk_UpdateAllMeasurements(bool scd_ok, bool bme_ok, bool flame_adc_ok, const SensorValues *values, const InternetWeather *weather)
```

Formats all current readings and transmits them in one batched Blynk HTTP update to reduce socket overhead.

1. Format all fixed-point sensor/weather values into decimal strings.
2. Build one bounded Blynk batch path containing all virtual-pin assignments.
3. Open one TCP connection and send one HTTP request rather than one connection per measurement.
4. Collect the response and report success only for HTTP 200.

IMPLEMENTATION NOTE

Flame transitions can trigger an immediate batch; normal telemetry is rate-limited to the configured ten-second interval.

4 Modification and debugging guide

Where to make common changes without breaking unrelated subsystems

Goal	Primary function(s)	Do not forget
Change dashboard layout	GUI_ShowWeatherDashboard(), LCD_Draw*(), LCD_DrawIcon()	Compose only in the back buffer and call LCD_PresentTextLayer() once per completed frame.
Change alarm limits	SystemAlarm_Evaluate(), AirQuality_Get()	Keep hysteresis/confirmation for noisy gas readings and preserve flame priority.
Change sample rates	Timing #defines + main()	Use unsigned HAL tick subtraction so wraparound remains safe.
Add a Blynk value	Blynk_UpdateAllMeasurements()	Assign a unique virtual pin and keep request construction bounded through BufferAppend().
Add a weather theme	QSPI_WeatherTheme, Weather_SelectTheme(), asset generator	Increment asset version, reinstall once, then disable installer mode.
Debug I^2C	SCD41_Read()/BME688_Read() and USART3 output	Do not replace the live display from background network/sensor diagnostics.
Retest SDRAM	Dedicated SDRAM diagnostic firmware	The retained SDRAM_Test() is destructive and must never run while GUI buffers are live.

4.1 Recommended debugging order

1. Confirm the failing subsystem's hardware power and pin configuration before changing code.
2. Use USART3 to capture exact HAL/AT responses; keep the dashboard active for user-facing state only.
3. Verify the producer first (sensor/network parser), then the cached data structure, then the GUI formatter.
4. For display corruption, verify SDRAM startup and QSPI asset CRCs before editing draw coordinates.
5. After any scheduling change, repeat flame detection while a network request is deliberately slow.

RELEASE CHECKLIST

Set Wi-Fi and Blynk credentials outside version control; set QSPI_ASSET_INSTALLER to 0 after successful installation; keep destructive memory tests excluded; compile with warnings enabled; verify sensors, alarms, weather, Blynk and the full 8 MiB SDRAM test before demonstration.

Appendix A Function count and coverage

Subsystem	Total	Active	Conditional
Startup, scheduler and fault handling	6	5	1
CubeMX peripheral initialization	10	10	0
External memory and QSPI assets	43	27	16
Display, drawing and GUI	30	30	0

Subsystem	Total	Active	Conditional
Environmental sensors	11	11	0
Flame and gas safety	5	5	0
Air-quality and alarm logic	6	6	0
ESP-01 and HTTP transport	9	9	0
Location, time and weather	16	16	0
Blynk telemetry and formatting	5	5	0
TOTAL	**141**	**124**	**17**

Coverage statement: all C function definitions found by brace-aware parsing of Core/Src/main.c and Core/Src/qspi_assets.c are represented in Section 2. Data-only generated asset modules contain no functions. STM32 HAL and Bosch BME68x library functions are dependencies and are not re-documented here.